

# Parallelizing Neural Network Training: Data Parallel vs Model Parallel

Anwith Goud Ganamaina, Vinaykumar Reddy Junuthula, Hrushikesh Rahul Reddy Tangala

CS Parallel Computing Project | Scientific Regression Data | PyTorch, MPI, OpenMP, HDF5

**Abstract**—This project implements and evaluates sequential, data-parallel, and model-parallel strategies for training a regression neural network on scientific-style HDF5 datasets. The study compares training time, throughput, epochs per second, communication overhead, memory usage, inter-rank data transfer, and final MSE. The terminal evidence uses a 1 GB dataset with one epoch and batch size 512. The results show that the sequential baseline is fastest for a one-epoch laptop run because it avoids distributed overhead; among parallel configurations, data parallel with 2 MPI ranks and 2 OpenMP threads gives the strongest parallel result. Model parallel is correct but becomes communication-bound, especially at 4 ranks, where 68.5% of total time is spent in communication.

**Index Terms**—Data parallelism, model parallelism, MPI, OpenMP, PyTorch, HDF5, neural network training, regression, HPC.

## I. INTRODUCTION

Large scientific datasets are common in areas such as particle physics, climate modeling, and scientific simulation. Training neural networks on these datasets can exceed the memory or processing limits of a single execution path. Parallel training is therefore important, but not every parallel strategy improves runtime. This project asks whether it is better to split the input data across workers or split the neural network layers across workers.

The implemented system compares three strategies: a sequential baseline, data parallel training, and model parallel training. Each run trains the same regression network and evaluates the same core performance metrics. The main goal is not only to check which strategy is faster, but also to explain why the difference occurs by examining communication time, inter-rank data movement, and memory usage.

## II. TEAM CONTRIBUTIONS

Anwith set up the environment, installed OpenMPI and dependencies, generated datasets, ran the experiments, collected screenshots, and prepared the final results. Vinay worked on the data-parallel design and helped analyze dataset splitting, DDP gradient synchronization, throughput, speedup, and communication overhead. Rahul worked on the model-parallel design and helped analyze layer splitting, activation/gradient transfer, communication overhead, memory usage, and scalability.

## III. SYSTEM DESIGN

The project uses PyTorch for model definition, automatic differentiation, and DistributedDataParallel. mpi4py provides explicit MPI process coordination and message passing. HDF5 is used for dataset storage because it supports chunked access and metadata at large scale.

OpenMP threads are used through PyTorch CPU kernels for matrix operations inside each rank.

The regression network uses 128 input features, four hidden layers with 512 units, ReLU activations, and 8 output targets. The network has about 853,000 parameters, which is large enough to produce meaningful compute work while remaining small enough to run on a laptop. The loss function is mean squared error because the task is scientific regression, not classification.

## IV. DATASET AND EXECUTION ENVIRONMENT

The synthetic dataset generator created three HDF5 datasets: approximately 1 GB, 5 GB, and 10 GB. The terminal screenshots in this report show the final comparison runs on the 1 GB file, containing 2,000,000 samples with 128 features and 8 targets. All terminal evidence was produced on a MacBook environment using OpenMPI 5.0.9, Python 3.13, PyTorch, mpi4py, h5py, NumPy, and Matplotlib.

## V. TRAINING STRATEGIES

In the sequential baseline, a single process trains the entire model with no MPI communication. This gives a reference runtime and confirms that the model and data pipeline are correct before introducing parallel complexity.

In data parallel training, each MPI rank keeps a full copy of the model and trains on a shard of the dataset. After backpropagation, gradients are synchronized across ranks. This strategy communicates gradients, but it avoids sending activations every batch.

In model parallel training, the neural network layers are divided across MPI ranks. During the forward pass, activations are sent from one rank to the next. During the backward pass, gradients are sent back through the pipeline. This allows the model to be split across ranks, but it introduces frequent communication every batch.

## VI. EXPERIMENTAL SETUP

The screenshot-supported experiment used the 1 GB dataset, one epoch, batch size 512, 4 hidden layers with 512 hidden units, parallel HDF5 I/O mode, Caliper off, and PAPI off. The configurations were sequential 1r x 1t, data parallel 2r x 2t, data parallel 4r x 2t, model parallel 2r x 2t, and model parallel 4r x 2t.

The main metrics reported are training time, epochs per second, compute time, communication time, MPI data transferred, I/O time, throughput, memory usage, final MSE, and memory per batch. Speedup is computed as sequential time divided by parallel time, and efficiency is computed as speedup divided by number of ranks.

## VII. EXPERIMENTAL RESULTS

Table I records the values from the terminal screenshots. Because the experiment is a one-epoch laptop run, the sequential baseline is fastest overall. Among the parallel runs, data parallel with 2 ranks and 2 threads is the best parallel configuration. Model parallel with 4 ranks is slowest because communication dominates its runtime.

TABLE I. Updated results from terminal screenshots (1 GB dataset, 1 epoch, batch size 512).

| Strategy       | Ranks | Thrds | Time(s) | Epochs/s | Throughput | Comm% | Comm MB   | Memory | Final MSE |
|----------------|-------|-------|---------|----------|------------|-------|-----------|--------|-----------|
| Sequential     | 1     | 1     | 12.3552 | 0.0809   | 161,876    | 0.0   | 0.00      | 2212.8 | 5.630362  |
| Data Parallel  | 2     | 2     | 14.1639 | 0.0706   | 141,204    | 0.0   | 12,786.17 | 1180.8 | 5.630282  |
| Data Parallel  | 4     | 2     | 16.2824 | 0.0614   | 122,832    | 0.0   | 12,779.62 | 462.8  | 5.630786  |
| Model Parallel | 2     | 2     | 14.5885 | 0.0685   | 137,095    | 44.7  | 7,873.03  | 2790.0 | 5.630368  |
| Model Parallel | 4     | 2     | 31.9677 | 0.0313   | 62,563     | 68.5  | 23,497.03 | 1304.9 | 5.630292  |

TABLE II. Derived speedup and efficiency from the same one-epoch measurements.

| Configuration        | Train Time(s) | Speedup | Efficiency | Communication % | Interpretation                                   |
|----------------------|---------------|---------|------------|-----------------|--------------------------------------------------|
| Sequential 1r_1t     | 12.3552       | 1.000x  | 1.000      | 0.0%            | Fastest overall in 1-epoch run                   |
| Data Parallel 2r_2t  | 14.1639       | 0.872x  | 0.436      | 0.0%            | Best parallel approach when communication is low |
| Data Parallel 4r_2t  | 16.2824       | 0.759x  | 0.190      | 0.0%            | Best parallel approach when communication is low |
| Model Parallel 2r_2t | 14.5885       | 0.847x  | 0.423      | 44.7%           | Communication-heavy; scalability is limited      |
| Model Parallel 4r_2t | 31.9677       | 0.386x  | 0.097      | 68.5%           | Communication-heavy; scalability is limited      |

## VIII. TERMINAL EVIDENCE SCREENSHOTS

The following screenshots are included as direct proof of the commands and results used in the report tables. Each screenshot shows the strategy, MPI ranks, OpenMP threads, dataset, training output, and saved metrics file.

```
=====
PARALLEL NN TRAINING: DATA PARALLEL vs MODEL PARALLEL
Task: REGRESSION (MSE Loss) | Scientific Data
=====
Strategy      : sequential
MPI Ranks     : 1
OMP Threads   : 1
Epochs       : 1
Batch Size    : 512
Model         : 4 hidden x 512
Dataset       : data/dataset_1gb.h5 (2,000,000 x 128 -> 8, 1038MB)
Source        : synthetic
I/O Mode      : parallel
Caliper       : off
PAPI          : off
=====
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/
ed memory won't be used.
super().__init__(loader)
[ [Sequential] Epoch 1/1 | MSE: 5.630362
=====
RESULTS
=====
Train Time    : 12.3552s
Epochs/s     : 0.0809
Compute       : 11.1210s (90.0%)
Communication : 0.0000s (0.0%)
Comm Data     : 0.00 MB transferred
I/O           : 0.0000s (0.0%)
Throughput    : 161876 samples/s
Memory        : 2212.8 MB
Final MSE     : 5.630362
Loss Improve  : 0.000000
Mem/Batch     : 4.3220 MB
=====
Saved: results/metrics_sequential_1r_1t_2000000s.json
```

Fig. 1. Sequential baseline terminal output.

```

=====
PARALLEL NN TRAINING: DATA PARALLEL vs MODEL PARALLEL
Task: REGRESSION (MSE Loss) | Scientific Data
=====
Strategy      : data_parallel
MPI Ranks     : 4
OMP Threads   : 2
Epochs       : 1
Batch Size    : 512
Model         : 4 hidden x 512
Dataset       : data/dataset_1gb.h5 (2,000,000 x 128 -> 8, 1038MB)
Source        : synthetic
I/O Mode      : parallel
Caliper       : off
PAPI          : off
=====
Rank 0: 500000 local samples
Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/t
d memory won't be used.
super().__init__(loader)
Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/t
d memory won't be used.
super().__init__(loader)
Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/t
d memory won't be used.
super().__init__(loader)
Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/t
d memory won't be used.
super().__init__(loader)
[DataParallel] Epoch 1/1 | MSE: 5.630786
=====
RESULTS
=====
Train Time    : 16.2824s
Epochs/s     : 0.0614
Compute       : 13.7707s (84.6%)
Communication  : 0.0011s (0.0%)
Comm Data     : 12779.62 MB transferred
I/O           : 0.7244s (4.3%)
Throughput    : 122832 samples/s
Memory        : 462.8 MB
Final MSE     : 5.630786
Loss Improve  : 0.000000
Mem/Batch     : 0.9039 MB
=====
Saved: results/metrics_data_parallel_4r_2t_200000s.json

```

Fig. 2. Data parallel, 4 ranks x 2 threads terminal output.

```

=====
PARALLEL NN TRAINING: DATA PARALLEL vs MODEL PARALLEL
Task: REGRESSION (MSE Loss) | Scientific Data
=====
Strategy      : model_parallel
MPI Ranks     : 4
OMP Threads   : 2
Epochs       : 1
Batch Size    : 512
Model         : 4 hidden x 512
Dataset       : data/dataset_1gb.h5 (2,000,000 x 128 -> 8, 1038MB)
Source        : synthetic
I/O Mode      : parallel
Caliper       : off
PAPI          : off
=====
[ModelParallel] Epoch 1/1 | MSE: 5.630292
=====
RESULTS
=====
Train Time    : 31.9677s
Epochs/s     : 0.0313
Compute       : 5.5952s (17.5%)
Communication  : 21.8967s (68.5%)
Comm Data     : 23497.03 MB transferred
I/O           : 0.2780s (0.9%)
Throughput    : 62563 samples/s
Memory        : 1304.9 MB
Final MSE     : 5.630292
Loss Improve  : 0.000000
Mem/Batch     : 2.5486 MB
=====
Saved: results/metrics_model_parallel_4r_2t_200000s.json

```

Fig. 3. Model parallel, 4 ranks x 2 threads terminal output.

```

=====
PARALLEL NN TRAINING: DATA PARALLEL vs MODEL PARALLEL
Task: REGRESSION (MSE Loss) | Scientific Data
=====
Strategy      : data_parallel
MPI Ranks     : 2
OMP Threads   : 2
Epochs       : 1
Batch Size    : 512
Model         : 4 hidden x 512
Dataset       : data/dataset_1gb.h5 (2,000,000 x 128 -> 8, 1038MB)
Source        : synthetic
I/O Mode      : parallel
Caliper       : off
PAPI          : off
=====
Rank 0: 1000000 local samples
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-package
ed memory won't be used.
super().__init__(loader)
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-package
ed memory won't be used.
super().__init__(loader)
[DataParallel] Epoch 1/1 | MSE: 5.630282

=====
RESULTS
=====
Train Time    : 14.1639s
Epochs/s     : 0.0706
Compute       : 11.5259s (81.4%)
Communication : 0.0013s (0.0%)
Comm Data     : 12786.17 MB transferred
I/O           : 0.5184s (3.5%)
Throughput    : 141204 samples/s
Memory        : 1180.8 MB
Final MSE     : 5.630282
Loss Improve  : 0.000000
Mem/Batch     : 2.3063 MB
=====
Saved: results/metrics_data_parallel_2r_2t_2000000s.json

```

Fig. 4. Data parallel, 2 ranks x 2 threads terminal output.

```

=====
PARALLEL NN TRAINING: DATA PARALLEL vs MODEL PARALLEL
Task: REGRESSION (MSE Loss) | Scientific Data
=====
Strategy      : model_parallel
MPI Ranks     : 2
OMP Threads   : 2
Epochs       : 1
Batch Size    : 512
Model         : 4 hidden x 512
Dataset       : data/dataset_1gb.h5 (2,000,000 x 128 -> 8, 1038MB)
Source        : synthetic
I/O Mode      : parallel
Caliper       : off
PAPI          : off
=====
[ModelParallel] Epoch 1/1 | MSE: 5.630368

=====
RESULTS
=====
Train Time    : 14.5885s
Epochs/s     : 0.0685
Compute       : 5.5569s (38.1%)
Communication : 6.5231s (44.7%)
Comm Data     : 7873.03 MB transferred
I/O           : 0.1211s (0.8%)
Throughput    : 137095 samples/s
Memory        : 2790.0 MB
Final MSE     : 5.630368
Loss Improve  : 0.000000
Mem/Batch     : 5.4492 MB
=====
Saved: results/metrics_model_parallel_2r_2t_2000000s.json

```

Fig. 5. Model parallel, 2 ranks x 2 threads terminal output.

## IX. UPDATED GRAPHICAL RESULTS

The following graphs are generated from the values in the terminal screenshots. They visualize throughput, overhead, scalability, memory usage, communication volume, and MSE for the one-epoch experiment.

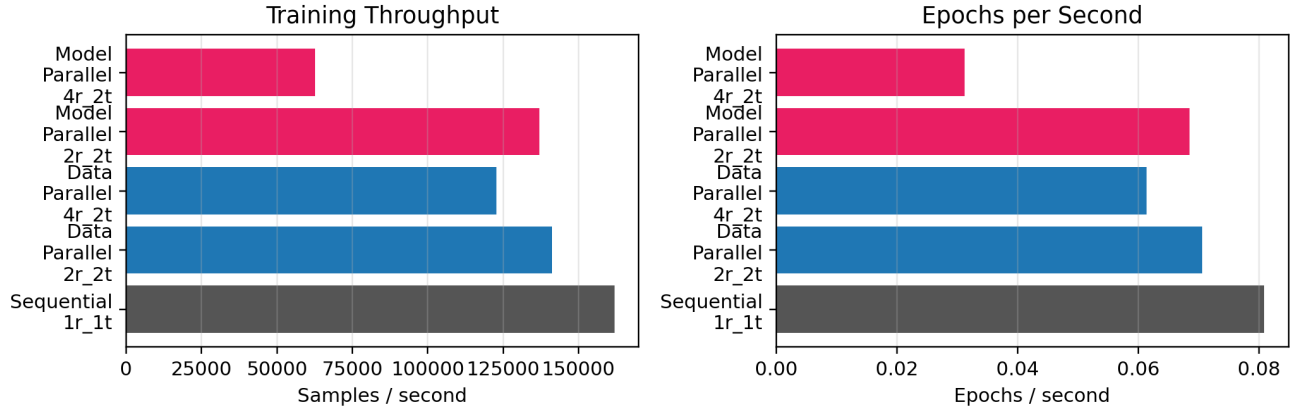


Fig. 6. Throughput and epochs per second for each configuration.

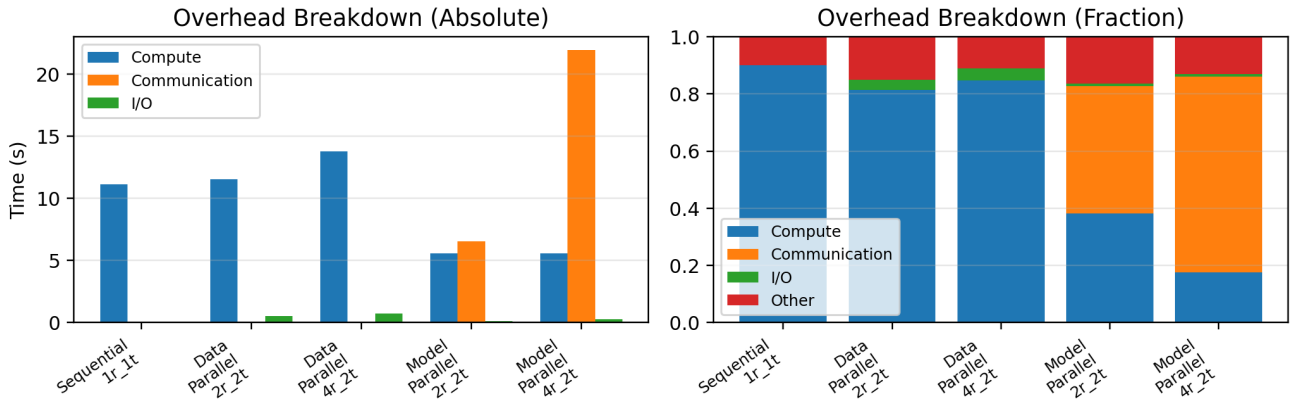


Fig. 7. Absolute and fractional runtime overhead breakdown.

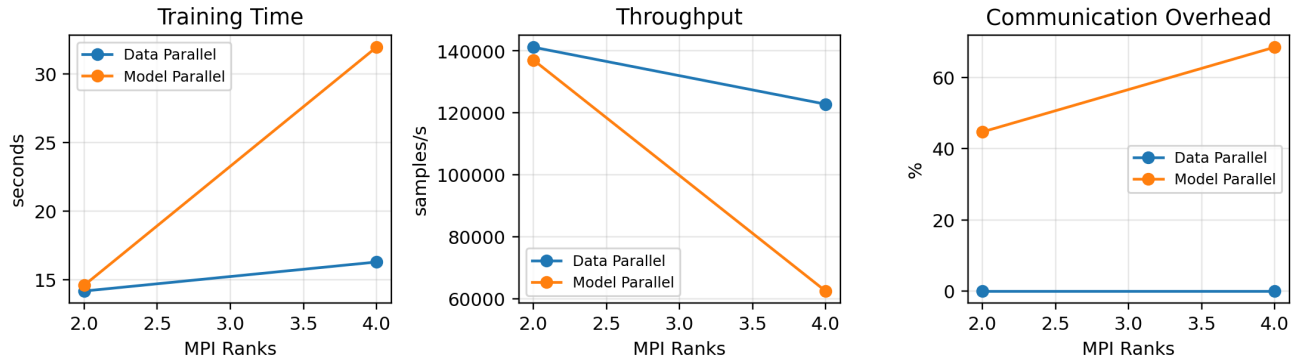


Fig. 8. Data parallel vs. model parallel trends as MPI ranks increase.

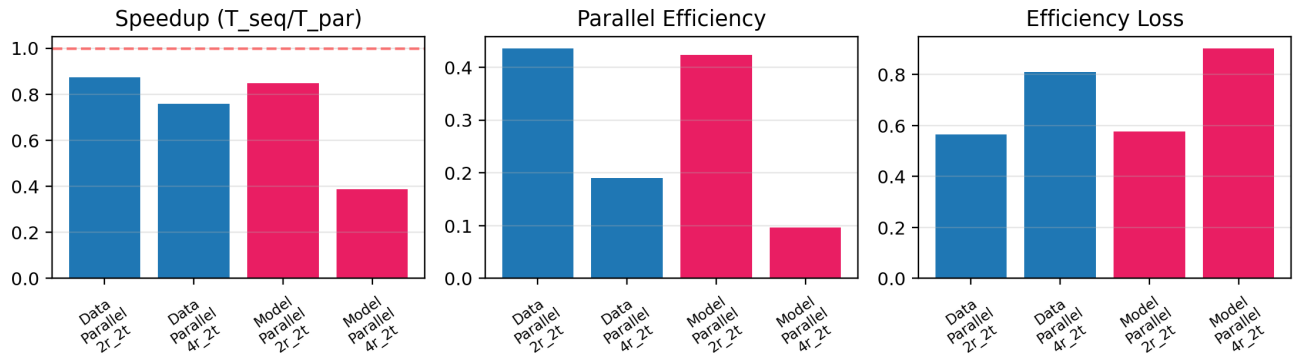


Fig. 9. Speedup, parallel efficiency, and efficiency loss relative to the sequential baseline.

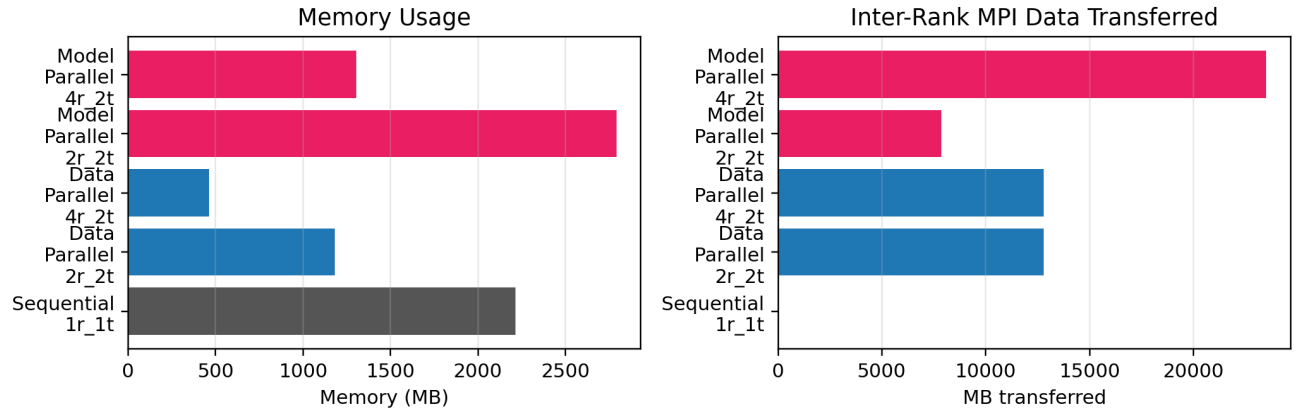


Fig. 10. Memory usage and inter-rank MPI data transferred.

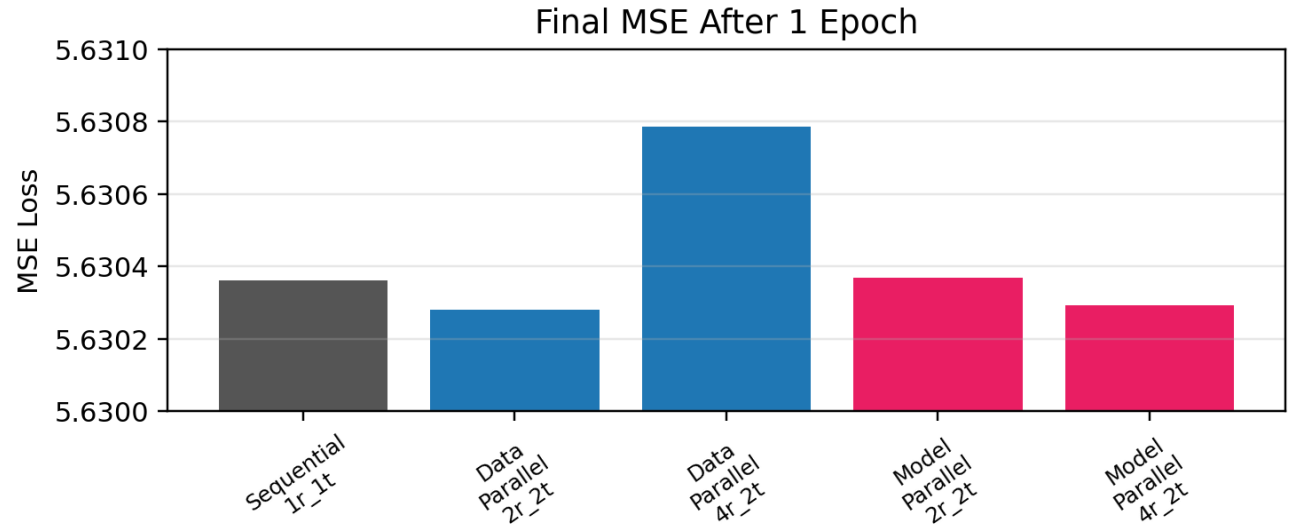


Fig. 11. Final MSE after one epoch for all configurations.



## X. RESULTS ANALYSIS

The terminal results show that the sequential baseline completed in 12.3552 seconds with 161,876 samples per second. This is the fastest overall result for the one-epoch experiment because it avoids distributed initialization and inter-process communication overhead. This does not mean that parallel training is useless; it shows that short runs on a single laptop can be dominated by setup and coordination cost.

Among the parallel configurations, data parallel with 2 ranks and 2 threads performed best. It completed in 14.1639 seconds and reached 141,204 samples per second. Its communication time was nearly zero as a percentage of runtime, even though the run transferred about 12,786 MB of gradient-related communication data. This shows that data parallelism is efficient when communication is well-managed and the number of ranks does not oversubscribe the system too heavily.

Data parallel with 4 ranks and 2 threads completed in 16.2824 seconds, which was slower than the 2-rank version in this one-epoch run. This is likely due to higher CPU and memory contention on the laptop. Four MPI ranks with two OpenMP threads each create more simultaneous work, and the benefit of additional ranks is not enough to overcome overhead in a short one-epoch test.

Model parallel with 2 ranks performed close to data parallel with 2 ranks in total runtime, but the internal breakdown shows a very different behavior. It spent 44.7% of runtime in communication. This is important because the approach may appear competitive in total time for 2 ranks, but the communication fraction shows it is already bottlenecked.

Model parallel with 4 ranks was the slowest configuration. It took 31.9677 seconds, achieved only 62,563 samples per second, and spent 68.5% of total runtime in communication. This confirms the main performance issue: adding more model-parallel stages increases the number of activation and gradient transfers between MPI ranks.

Correctness is supported by the final MSE values. All configurations finish near 5.630, meaning the parallel implementations are learning essentially the same regression function as the sequential baseline. The differences are therefore performance differences, not correctness failures.

## XI. DISCUSSION

The experiment demonstrates a central HPC lesson: parallelism is useful only when the added communication and coordination overhead is smaller than the compute benefit. Data parallelism communicates gradients but keeps

most work local. Model parallelism must communicate activations and gradients every batch, which becomes expensive as the number of pipeline stages increases.

For this one-epoch laptop experiment, the best overall runtime is sequential, while the best parallel configuration is data parallel with 2 ranks and 2 threads. For longer runs, larger datasets, or multi-node environments, data parallelism is expected to become more useful because the fixed overhead becomes smaller relative to the amount of computation. Model parallelism would be more appropriate for models too large to fit on one device, but it is not the best choice for this model on a single CPU-based system.

## XII. CONCLUSION

This project successfully implemented and compared sequential, data-parallel, and model-parallel neural network training for a scientific regression workload. The evidence shows that all methods are correct because final MSE values remain very close. The performance results show that sequential training is fastest for the one-epoch laptop run, but data parallel with 2 ranks and 2 threads is the best parallel strategy. Model parallel training becomes communication-bound, especially at 4 ranks, and therefore performs worst in this experiment.

The main conclusion is that for this workload and hardware, splitting the data is better than splitting the model. Data parallelism keeps communication low and maintains high throughput, while model parallelism introduces frequent activation and gradient transfers that dominate runtime.

## ACKNOWLEDGMENT

This report was prepared by Anwith, Vinay, and Rahul as part of the Parallel Computing project on neural network training using MPI, OpenMP, PyTorch, and HDF5.

## REFERENCES

- [1] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2019.
- [2] PyTorch, "DistributedDataParallel," PyTorch Documentation. [Online]. Available: <https://docs.pytorch.org/docs/stable/notes/ddp.html>. Accessed: May 8, 2026.
- [3] PyTorch, "Getting Started with Distributed Data Parallel," PyTorch Tutorials. [Online]. Available: [https://docs.pytorch.org/tutorials/intermediate/ddp\\_tutorial.html](https://docs.pytorch.org/tutorials/intermediate/ddp_tutorial.html). Accessed: May 8, 2026.
- [4] Message Passing Interface Forum, "MPI: A Message-Passing Interface Standard," Version 4.1, 2023. [Online]. Available: <https://www.mpi-forum.org/docs/>. Accessed: May 8, 2026.
- [5] L. Dalcin, R. R. Paz, P. A. Kler, and A. Cosimo, "Parallel Distributed Computing Using Python," Advances in Water Resources, vol. 34, no. 9, pp. 1124-1139, 2011.
- [6] L. Dalcin, "MPI for Python," mpi4py Documentation. [Online]. Available: <https://mpi4py.readthedocs.io/>. Accessed: May 8, 2026.
- [7] Open MPI Project, "Open MPI Documentation." [Online]. Available: <https://docs.open-mpi.org/>. Accessed: May 8, 2026.



- [8] The HDF Group, "HDF5 User Documentation." [Online]. Available: <https://docs.hdfgroup.org/>. Accessed: May 8, 2026.
- [9] h5py Developers, "HDF5 for Python," h5py Documentation. [Online]. Available: <https://docs.h5py.org/>. Accessed: May 8, 2026.
- [10] h5py Developers, "Parallel HDF5," h5py Documentation. [Online]. Available: <https://docs.h5py.org/en/stable/mpi.html>. Accessed: May 8, 2026.
- [11] Lawrence Livermore National Laboratory, "Caliper: A Performance Analysis Toolbox in a Library." [Online]. Available: <https://software.llnl.gov/Caliper/>. Accessed: May 8, 2026.
- [12] P. J. Mucci, S. Browne, C. Deane, and G. Ho, "PAPI: A Portable Interface to Hardware Performance Counters," in Proc. Department of Defense HPCMP Users Group Conference, 1999.
- [13] Lawrence Livermore National Laboratory, "PAPI: Performance Application Programming Interface." [Online]. Available: <https://hpc.llnl.gov/software/development-environment-software/papi-performance-application-programming-interface>. Accessed: May 8, 2026.
- [14] P. Baldi, P. Sadowski, and D. Whiteson, "Searching for Exotic Particles in High-Energy Physics with Deep Learning," Nature Communications, vol. 5, 2014.
- [15] UCI Machine Learning Repository, "HIGGS Data Set." [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/HIGGS>. Accessed: May 8, 2026.
- [16] UCI Machine Learning Repository, "YearPredictionMSD Data Set." [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/yearpredictionmsd>. Accessed: May 8, 2026.
- [17] W. McKinney, "Data Structures for Statistical Computing in Python," in Proc. 9th Python in Science Conference (SciPy), 2010, pp. 56-61.
- [18] C. R. Harris et al., "Array Programming with NumPy," Nature, vol. 585, pp. 357-362, 2020.